Автономная некоммерческая организация высшего образования
«Университет Иннополис»
ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»
FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»
Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»
Field of Study:
«Data Science and Artificial Intelligence»

Тема	**Генеративная адаптация веб-интерфейсов под цели пользователя в режиме реального времени**
Topic	**User-Aligned Web Interfaces: Just-in-Time Generative Adaptation of Existing UIs**

Работу выполнил / *Prepared by*	**Левада Андрей Романович / Andrey Levada**	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Лукманов Рустам Абубакирович / Rustam Lukmanov**	подпись / signature
Консультанты *Consultants*		подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **8**

1.1 Background 8

1.2 Internet Shaper 9

2 Related work **11**

3 Implementation **17**

3.1 Design Goals 17

3.2 Baseline Limitations 18

3.3 Internet Shaper 19

3.4 Perception 21

3.4.1 DOM Compression Algorithm 22

3.4.2 Selective Drill-Down 24

3.4.3 Alternatives 25

3.5 Action 25

3.5.1 Rules Engine 26

3.6 Agent Workflow 28

3.7 The Browser Extension 29

3.8 Security Concerns 30

4 Methods **32**

4.1 DOM Compression Evaluation 32

4.1.1 Source and filtering 32

4.1.2 Capture and quality control 33

4.1.3 Compression measurements 33

4.2 Task Synthesis 34

4.3 Ablation Pipelines 35

4.3.1 Local inference hardware 35

4.3.2 Human evaluation test stand 36

4.3.3 Sample filtering and scope 36

4.4 AI Usage Disclosure 37

5 Results **38**

5.1 DOM compression 38

5.1.1 Corpus and procedure 38

5.1.2 Token counts 39

5.1.3 Character counts and comment overhead 40

5.1.4 Implications for the prototype 40

5.2 Automated ablation corpus 41

5.3 Pairwise preference evaluation 42

5.4 Extension cases outside the corpus 43

6 Discussion **46**

6.1 Emergent behaviors 47

7 Conclusion **48**

7.1 Limitations 49

8 Data and Code Availability **50**

Bibliography cited **51**

List of Figures

3.1 Internet Shaper architecture overview 20

3.2 Single-child wrapper collapse 23

3.3 Repeated-sibling truncation 23

5.1 System in action, example 1. Google Images — Prompt: «remove the text under images». 44

5.2 System in action, example 2. Gmail — Prompt: «style the page in Minecraft theme». 44

5.3 System in action, example 3. Substack — Prompt: «replace the Up next block with an audio player from Spotify». 45

List of Tables

3.1 Estimated token count of popular web page DOMs 19

4.1 Agent ablation conditions 35

5.1 Token counts after DOM compression 39

5.2 Character counts after DOM compression 40

5.3 Median inference time by ablation pipeline 41

5.4 Screenshot task completion vs. original page 42

5.5 Human preference scores for component ablation 43

Abstract

Commercial web interfaces often misalign with individual users' goals: designers cannot anticipate every task, and gray or dark patterns can prioritize business metrics over usability. Adaptive interfaces that developers build at design time don't help users reshape third-party sites they already use. We present Internet Shaper, a browser-extension-hosted agentic system that adapts live web pages from natural-language requests without access to site source code. The system separates *perception* from *action*: a DOM compression algorithm gives a large language model a compact structural view of the page, and a rules engine persists selector-bound JavaScript that re-applies on reload and on DOM mutation. We evaluate compression on 73 snapshots from high-traffic domains, and collect preliminary human preferences on 15 samples. Compression reduces median visible DOM size by roughly 16×; the Internet Shaper system processes user requests about five times faster than a baseline on the same tasks. Screenshot-based judgments favor adapted pages over originals, but quality parity between the full system and the baseline remains inconclusive at this sample size.

Chapter 1

Introduction

1.1 Background

Designers and developers build user interfaces to make interactions with tech simpler and easier. Designers generally strive to make the UI as usable as possible for each user [1], [2]; nevertheless, two structural limitations undermine this alignment:

- Intrinsically, designers cannot feasibly plan interfaces for every single user's specific needs and jobs. With constrained resources, even good interfaces are usually not perfect for each user.
- On the other, more bleak side, gray and dark patterns systematically misalign system and user goals. This happens when business and user goals diverge, and the interface ends up being less helpful and more intrusive [3], [4], [5].

These two problems make interfaces misaligned with users' needs — undermining the core of design practice. This is the problem we begin to tackle with this thesis.

User interfaces can be made adaptive and customizable to cater to specific users' needs. However, customization and especially automatic adaptation of UIs

require the developer or designer to implement them. As we see in the industry, these methods are not widely adopted except for select adaptivity dimensions — iOS and Android apps commonly react to dynamic type accessibility settings by changing the layout of the app to better display content with larger text, and websites are almost always made responsive to screen sizes. But as a way of mitigating misaligned interfaces, the adaptive-interface approach is not really used.

Few works take a different approach and try to make interfaces malleable by end users [6], [7]. In this thesis we build upon the ideas they introduce. We want to realize the vision of individually tailored user interfaces, shared by many UX practitioners [8], [9], made possible by recent advances in LLM capabilities.

1.2 Internet Shaper

We design, build, and evaluate Internet Shaper — an agentic system wrapped in a browser extension that can change web pages from natural-language requests. It works directly on the page in the user's browser and does not need access to the website's source code. These changes are persistent across sessions and are powerful enough to restyle, hide, or change elements and whole layouts, or even bring new, although limited, functionality to the website.

In our evaluation we show how this system enables user-controlled adaptations of existing web interfaces and can function as a way for users to actively realign UIs they use with their needs and interests.

Our contributions are as follows:

1. Internet Shaper as a whole and its two critical components: a DOM compression algorithm for perceiving web pages, and the Rules Engine for applying changes to the webpage in a persistent way
2. A pipeline that can create datasets with user-sided natural-language edit requests grounded in user personas and jobs
3. An evaluation of Internet Shaper on a dataset gathered from that pipeline

Chapter 2

Related work

HCI practitioners have long pursued interfaces that adapt to individual users or give users direct control over customization. [10] reviews 127 generative-UI publications and argues the field is shifting from designer-only tooling toward interfaces generated or reshaped around individual tasks. [8] makes a similar case in industry terms, describing generative UI as a move from designing for many users to tailoring outcomes for one.

We distinguish two adaptation regimes along when and who initiates change. In *design-time* adaptation, developers plan and implement adaptive or customizable behavior while the application is still being built; at use time the shipped product either adapts on its own or offers customization the developers designed into it. In *just-in-time* adaptation, the user encounters a fixed interface — typically a third-party page that was not built to change — and initiates modification in response to an articulated goal. Internet Shaper targets the second regime.

The clearest example of a design-time adaptation method is ReLay [11], a browser probe that infers browsing intent and automatically adjusts information

hierarchy, granularity, and session ordering while the user reads. In a two-phase study, participants accepted these in-situ changes when they remained transparent, consistent, and easily reversible. ReLay shows that intent-responsive layout can improve browsing, but it still depends on a researcher-built adaptive shell.

That design-time constraint is the central limitation of existing methods. If developers never implement adaptive or customizable behavior, end users cannot change the interface at all. In practice, such features are rarely prioritized outside a few well-resourced dimensions such as responsive layout or platform accessibility settings [12]. Adaptive interfaces therefore do little to help users realign hostile or misaligned third-party sites they already depend on.

A second line of work lets users reshape interfaces around their tasks, but still inside systems that researchers or product authors control. [13] introduces malleable overview–detail interfaces: end users can change content, composition, and layout of a common UI pattern, including AI-assisted attribute manipulation between views. The paper demonstrates demand for task-aligned presentation, but the customization machinery is built into author-controlled design probes, not retrofitted onto opaque third-party DOM. [14] generates interfaces from task-driven data models that users can extend through natural language and direct manipulation. It shares our interest in interfaces that follow the user's task, but requires authors to implement the generative data-model layer up front.

[15] shows that a single LLM, with prompting alone, can support diverse conversational interactions with mobile UIs without task-specific training datasets.

It establishes NL as a viable UI control channel, but targets mobile applications with developer-provided screen representations rather than arbitrary web pages accessed through a browser extension. [16] gives designers a graphical web inspector that lowers the cost of style edits compared to browser developer tools. It improves professional design workflows during creation, not end-user just-in-time adaptation of live third-party sites at use time. [17] presents an IDE-embedded AI companion that helps students build interactive portfolio UIs from natural-language prompts. It shows generative reshaping from articulated intent, but in an authoring environment the user controls rather than on websites they merely visit. [18] benchmarks vision-language models that operate design tools through sequential tool invocations. It evaluates model capacity to manipulate UI design files, which is orthogonal to transforming rendered third-party pages in the user's browser.

A related malleable-web lineage treats live pages as editable data rather than fixed surfaces. [6] introduces Wildcard, a browser extension that syncs a spreadsheet-like table to scraped website data so end-user table edits propagate back to the live page. It is foundational evidence that production websites can be customized without source access, but its spreadsheet-and-formula model is narrower than open-ended natural-language goals and does not provide our cross-session rules engine. [19] unifies web extraction and augmentation in one spreadsheet formula language with programming-by-demonstration on DOM elements. It reduces the separation between reading and changing a page, yet still asks users to think in formulas rather than goals and lacks persistent selector-bound JavaScript rules. [7] lets end users

create, extend, and repair site adapters by demonstration instead of relying on pre-built programmer-written adapters. That direction is directly relevant to reducing manual wrapper authorship; Internet Shaper replaces demonstrated adapters with LLM perception over a compressed DOM. [20] presents Vegemite, which combines spreadsheet programming and demonstration to let end users build cross-site web mashups. It established early malleable-web programming, but targets mashup composition across sites rather than durable transformation of one page the user returns to repeatedly.

Far fewer systems give end users tools to adapt existing web interfaces they do not own. This end-user web adaptation literature is the closest prior work to Internet Shaper.

[21] presents Sifter, a browser extension that auto-scrapes list data and adds in-page sort and filter controls as if they were native site features. It demonstrates the scrape-then-augment pattern on live DOM, but limits scope to list filtering and does not support open-ended natural-language goals or persistent rules. [22] defines Sticklet, a declarative JavaScript grammar for end-user web augmentation aimed at both producers and consumers of modifications. It addresses the same browser-side augmentation problem we face, but relies on manually authored augmentation scripts rather than LLM-generated rules.

[23] describes WebMakeup, a Chrome extension that lets users attach widgets to clicked DOM nodes and rearrange page fragments visually. It shows direct page modding without natural language, but its visual editing model is fragile on changing

sites and does not support conditional logic over runtime content. [24] presents MAWA, a mobile Firefox extension with a visual DSL for removing and moving page content to improve mobile reading. It confirms that extension-based DOM rewriting is practical on real sites, but focuses on layout-oriented mobile augmentation rather than goal-driven NL adaptation with logic rules. [25] enables crowdsourced web page adaptation through direct manipulation of layout — moving, resizing, hiding blocks, and changing typography — for individual viewing conditions. It supports layout personalization without scripting, but produces shared crowd variants rather than private, hostname-scoped rules the individual user owns. [26] personalizes websites continuously using selector–template pairs: regex-like selectors over interaction logs paired with JavaScript template skeletons, validated in a long-term field study.

[27] presents Stylette, a browser extension that maps natural-language styling goals to CSS property palettes using an LLM and a large web-component corpus. It is our closest contemporary peer because it combines extension deployment, live DOM access, natural language, and an LLM; however, it focuses on CSS appearance, does not persist behavioral rules across reloads in the way our engine does, and cannot express conditional logic such as hiding items based on parsed page content. We therefore build upon the limitations this paper acknowledges.

Current end-user adaptation methods remain limited relative to our goals. Changes are often ephemeral, scoped to appearance or layout, or tied to manually authored adapters rather than open-ended user goals [21], [25], [26], [27]. Like

this prior work, Internet Shaper is deployed as a browser extension that reads the live DOM. We extend the paradigm with LLM-driven perception over compressed page structure and a rules engine that persists selector-bound JavaScript, including conditional logic that CSS-only or one-shot edits cannot express.

Chapter 3

Implementation

This section describes how we designed and built the agentic system at the core of Internet Shaper. We begin from the user-side adaptation problem, explain why a naive read-and-edit baseline is not sufficient, and present the perception and action components that address those failures. The perception and action components are implemented as LLM tools plus a rules engine; the browser extension provides capture, storage, and execution. Source code, experiment pipelines, evaluation artifacts, and analysis logs are available in the public repository at github.com/andrewlevada/internet-shaper-thesis-2026 (Section 8).

3.1 Design Goals

As introduced above, interfaces are rarely perfect for every user. Even well-designed products leave gaps between a person's actual tasks and what the UI optimizes for. In the worst cases, gray and dark patterns deliberately misalign business incentives with user goals, making the interface harder to use for the tasks the user actually cares about.

Most adaptive interface research assumes developers build adaptive or customizable behavior into the application itself. Our approach is different: it is *user-sided*. The user adapts software they already use, without any cooperation from the people who built it. The system operates on pages that were not designed to be modified.

To reach those pages, we host the prototype in a browser extension. Extensions are uniquely positioned to read and write the DOM of any open tab. The page's structure and content are available to the system as HTML — the same representation the browser uses to render what the user sees. This gives us freedom that is miles ahead of other platforms.

Any adaptation system that changes the UI performs two sequential steps conceptually. First, it must somehow *read* the page: capture context, reason about structure, or even decompose the user's request into concrete targets. Second, it must *apply* changes to transform the page into a desired state. These steps are both minimal and required. Throughout this paper we call these two parts *perception* and *action*.

3.2 Baseline Limitations

The straightforward design is to read the full page HTML into the model, then apply direct text or DOM edits. Two practical limits make this baseline infeasible for real applications.

DOM snapshots from real websites are far larger than model context windows allow. The compression study in Section 5.1 measures 73 snapshots from popular domains (corpus protocol in Section 4.1). Even after removing invisible subtrees,

median visible DOM size remains above 100k tokens; raw snapshots reach 240k tokens at the median and over one million at the maximum (TABLE 3.1).

TABLE 3.1
Estimated token count of popular web page DOMs — Approximate token counts over 73 page snapshots from the top 25 traffic domains; counted with tiktoken `o200k_base`.

Stage	Mean (tok)	Median (tok)	Max (tok)
Raw DOM	339,253	240,918	1,338,122
Visible DOM	162,806	107,851	1,028,594

It is widely known that over-saturated contexts produce worse results in general task performance on LLMs [28], [29], [30]. Passing the full DOM on every request is therefore not viable: many pages would not fit, and those that do consume most of the context budget before any reasoning begins.

Even when a DOM fits, most of its HTML carries no information relevant to a given adaptation request — build-tool comments, framework wrapper elements, repeated list items, and design-system class tokens. A baseline that reads everything forces the model to filter noise on every turn.

On the action side, direct edit instructions do not survive a page reload. Without persistent storage, the system would need to re-run the full read-and-edit pipeline every time the user opens the page, relying on prompt caching or identical model outputs to reproduce the same result. That is slow, costly, and brittle on dynamic single-page applications where content loads after the initial render.

3.3 Internet Shaper

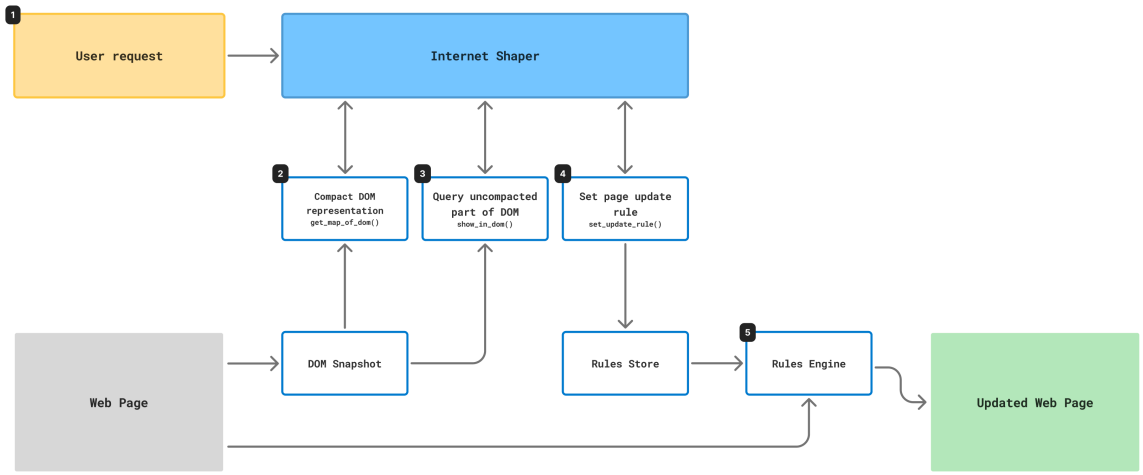

Fig. 3.1. Internet Shaper architecture overview — Agentic core with browser extension wrapper.

Internet Shaper is an agentic system that makes user-side adaptation of web UIs possible with two specialized components.

Perception is driven by a *DOM compression algorithm* that gives the model a compact view of the page with the relative structure of elements fully preserved, and a way to retrieve local detail when the overview is not enough.

Action is powered by the *Rules Engine*. Instead of direct edits, it records persistent update rules — selector-bound JavaScript that is re-applied on every load and on DOM mutation.

The LLM interacts with both components through tools in a fixed explore-then-act loop. It never writes to the live page directly; it reads from a snapshot frozen at request time and appends rules to a store.

Fig. 3.1 shows the end-to-end workflow inside the browser extension. The numbered steps proceed as follows:

1. The user writes a natural-language adaptation request.
2. The agent calls `get_map_of_dom()` and receives a compact structural map of a DOM snapshot captured at request time.
3. When the map is not enough, the agent calls `show_in_dom()` to inspect local detail from the original snapshot.
4. The agent calls `set_update_rule()` to record persistent, selector-bound JavaScript in the rules store — it does not edit the live page during the loop.
5. After the loop finishes, the rules engine applies stored rules to the live page on load and on DOM mutation, yielding the updated page.

3.4 Perception

An ideal perception interface must meet two criteria that the read-all baseline cannot (Section 3.2):

- *Scale*. The representation must fit within practical context limits even though median visible DOMs already exceed 100k tokens (TABLE 3.1).
- *Signal density*. It must shed the request-irrelevant bulk of production HTML — invisible markup, framework boilerplate, wrapper chains, and repetitive siblings — while preserving enough structure to locate and reason about adaptation targets.

Perception is implemented as a single DOM compression pipeline exposed through two complementary tools on the same captured snapshot: `get_map_of_dom` returns a site-wide structural map; `show_in_dom` retrieves local detail from the uncompressed snapshot when the map is not enough.

3.4.1 DOM Compression Algorithm

The compression pipeline operates on the visible DOM captured at request time (subtrees with `display: none`, `visibility: hidden`, or `opacity: 0` are already removed during capture, as in the evaluation corpus). `get_map_of_dom` applies the following steps in fixed order:

1. *Remove non-structural markup* — drop `head`, `script`, `link`, `style`, and `noscript` elements; build-tool HTML comments; and SVG path data (keeping each `svg` tag and its `title` for accessibility context).
2. *Attribute filtering* — each element keeps only `class`, `id`, `role`, `aria-label`, `label`, `alt`, `type`, `name`, `placeholder`, `value`, and `data-*` attributes; all others (including `href`, `src`, and inline styles) are dropped. Empty attribute values are removed afterward.
3. *High-frequency class removal* — class tokens appearing on more than 5% of elements are stripped tree-wide. The assumption is that very common classes are design-system scaffolding rather than component identifiers.
4. *Single-child wrapper collapse* — chains of elements with exactly one element child and no significant direct text are flattened: intermediate nodes are removed and replaced by an HTML comment recording how many wrappers were collapsed and which class or attribute tokens were lost. Chains stop at leaf elements or at nodes that carry inline text.

Fig. 3.2. Single-child wrapper collapse — Before (left) and after (right): three nested `div` wrappers around a button become `<!-- -3 wrappers -->` while the button and unrelated siblings are kept.

5. *Repeated-sibling truncation* — among groups of three or more consecutive siblings deemed structurally similar, only the first is kept; the rest are removed and annotated with a comment giving the truncated count. Similarity requires matching tag name and `id`, approximately matching class tokens (symmetric-difference thresholds scale with class-list length), and approximately matching sequences of immediate child tag names (compared via Levenshtein distance with length-dependent tolerances).

Fig. 3.3. Repeated-sibling truncation — Before (left) and after (right): three structurally similar `li` items become one representative plus `<!-- -2 siblings -->`.

6. *Whitespace normalization* — insignificant text nodes are removed and remaining text is collapsed to single spaces, except inside `script`, `style`, `pre`, and `textarea` ancestors.

The result is a compact HTML map that preserves relative hierarchy and enough identifying tokens to locate targets, at the cost of omitting repeated list items and decorative wrapper depth. On outlier pages the heuristics may barely shrink the tree (Section 5.1), and when a map still exceeds provider limits the production gateway truncates tool output at roughly 96k characters — the model must then rely on `show_in_dom` for the missing detail.

3.4.2 Selective Drill-Down

Because compaction is intentionally lossy for scale, the agent can call `show_in_dom(query_selector, depth)` on the *original* snapshot — not the compressed map. The tool resolves the selector against the full captured HTML, clones the matched subtree, and prunes descendants beyond a configurable depth (default three element levels below the matched node). At the depth boundary, nested elements are replaced by a comment of the form `<!-- -N children -->`, while direct text on retained nodes is kept intact.

This complements the map in two ways. First, it restores attributes and child structure removed or summarized during compaction — for example, `href` values, inline styles, or the second item in a truncated sibling group. Second, it bounds local context: the agent requests only the subtree it needs rather than reverting to a full-

DOM read. Increasing `depth` trades token cost for completeness when a rule must inspect deep descendants.

3.4.3 Alternatives

We also considered alternative comprehension strategies from the web-agent literature. Following [31]'s classification into text-based, screenshot-based, and multimodal approaches, screenshot-only input is incompatible with our action model, which targets elements via CSS selectors and JavaScript. The accessibility tree preserves semantics for assistive technologies but strips layout detail users often want to change [30]. Task-specific DOM pruning — as in Prune4Web [32] and related DOM-aware summarization methods [33] — suits localized edits, but many adaptation requests are global: restyling a feed, suppressing a class of distractions, or restructuring layout. Hierarchical observation summarization [28] and trajectory-based prompting [29] similarly optimize for navigation episodes rather than durable UI transformation. We therefore compress the full visible page structurally rather than pruning to a task-specific subset. We believe a combined approach of DOM compression together with pruning might work best, but this is out of scope for this thesis.

3.5 Action

An ideal action interface must meet two criteria that one-shot DOM edits cannot (Section 3.2):

- *Persistence*. Changes must survive reload and re-apply on dynamic pages without re-running the full perception pipeline or depending on identical model outputs each visit.
- *Expressiveness*. The mechanism must cover the same edit space direct manipulation allows — styling, interaction changes, and conditional logic over runtime content — not merely the subset CSS can hide or restyle.

Example. "Show only videos that are longer than 40 min" on a YouTube playlist page. A persistent rule must read duration text from each item and conditionally hide non-matching entries — logic that CSS and one-shot text replacements cannot express.

We considered a custom transformation language and generated CSS before settling on JavaScript. Frontier models already generate JS reliably for DOM manipulation [34], and JS covers the conditional logic CSS cannot. A custom language would add parsing cost without clear benefit.

3.5.1 Rules Engine

Internet Shaper uses update rules. An *update rule* is a persistent, selector-bound transformation stored as a small record: a human-readable `label` (shown in the rule manager UI), a CSS `query_selector`, and a `logic` string of JavaScript. Rules are scoped to the page hostname and persisted in extension storage; on each visit the extension loads the stored set and applies it before the user interacts with the page.

Example. For the YouTube playlist request in the expressiveness example above, the agent might emit a rule like the following:

1. **label:** `Hide videos under 40 min`
2. **query_selector:** `ytd-playlist-video-renderer`
3. **logic:**

```
const duration = element.querySelector('span.ytd-thumbnail-
overlay-time-status-renderer')?.textContent?.trim();
if (!duration) return;
const parts = duration.split(':').map(Number);
const minutes = parts.length === 3 ? parts[0] * 60 + parts[1] :
parts[0];
if (minutes < 40) element.style.display = 'none';
```

The LLM never executes rule logic during the agent loop. Instead, the `set_update_rule` tool appends a rule to an in-memory list. The tool schema exposes exactly the three fields above and documents the execution contract: the logic runs once per matched element with only an `element` parameter in scope — no `window`, `document`, or other globals. The system prompt further requires idempotent logic (repeated application must converge to the same state), deterministic side effects (set absolute values rather than toggle or append), and graceful handling of not-yet-loaded child content (early return when a value is missing, relying on re-application once content appears). Conditional hiding should prefer `element.style.display = 'none'` over `element.remove()` so the rule can still run when descendants populate.

After the agent loop completes, the new rules are merged into storage and passed to `applyRules`, which injects an applier function into the page's main JavaScript world via the extension's RPC layer. For each enabled rule, the engine queries `document.querySelectorAll`, wraps the logic as `(function(element) { ... })`, and compiles it through a Trusted Types policy where the host page requires one (needed on strict Content Security Policy sites such as YouTube). Each matching element is processed at most once per rule via a per-selector weak set; a `MutationObserver` on `document.body` re-applies rules to newly inserted nodes, and a child observer debounces re-runs when descendants of a matched element change — but disconnects after five seconds, so very late-loading content may never be transformed. Sites with strict CSP or Trusted Types allowlists that block our policy name may still reject dynamic rule compilation (Section 7.1).

From the model's perspective, a rule is therefore a declarative target (`query_selector`) plus imperative body (`logic`); from the runtime's perspective, it is a recurring DOM transformation that survives reloads and dynamic updates without re-invoking the LLM.

3.6 Agent Workflow

The system prompt enforces a strict perception-before-action sequence on every user request, following the tool-use agent pattern now common in LLM systems [35]. When the agent loop starts, the extension captures a DOM snapshot (`document.documentElement.outerHTML`) and passes it to the tool executor; all

perception tools read from this frozen copy rather than the live page. The model then follows this sequence:

1. Call `get_map_of_dom()` before any other tool.
2. Identify candidate elements relevant to the user's request from the returned map.
3. Call `show_in_dom(query_selector, depth)` when selector construction or local structure requires detail absent from the map — for example, distinguishing two elements that collapsed to the same outline, or reading `data-*` values needed for a conditional rule.
4. Emit one or more `set_update_rule(label, query_selector, logic)` calls.
5. Reply to the user when no further tools are needed; the loop continues until the model produces a message without tool calls or the stop reason is `end_turn`, at which point the collected rules are persisted and applied to the live page.

This ordering prevents the model from committing to selectors before it has a structural overview, while still allowing targeted inspection where the compressed map is insufficient. Because all perception tools read from the frozen snapshot, the agent cannot observe live DOM updates or the effect of rules during the loop — only the pre-request page state. `get_map_of_dom` and `get_dom` may each be called at most once per request; repeat calls return the cached result rather than a fresh capture. Prompt caching is applied to the system prompt and the `get_map_of_dom` result — typically the largest context item — so multi-turn exploration stays economical.

3.7 The Browser Extension

The agentic core is hosted in a Chromium browser extension — the prototype targets Chrome-compatible browsers only and is not ported to Firefox or Safari. It captures snapshots, hosts the LLM loop, stores rules in `localStorage`, and applies them in the page's main JavaScript world. Extensions can read and write any tab's DOM without site cooperation, which satisfies the deployment constraint from Section 3.1, but they add no algorithmic behavior beyond capture, RPC, and rule injection.

Browser extensions run code in isolated worlds that cannot call each other directly: content scripts, a background service worker, and the page's main JavaScript context. Fiber, a small framework built for this project, wraps Chrome APIs in an RPC proxy so calling `executeInMainWorld` from a content script behaves like a local function while messages cross process boundaries. Trusted Types policies registered in the main world allow dynamic compilation of rule logic on strict Content Security Policy sites such as YouTube.

The overlay UI — prompt input, rule manager, status — is implemented with Lit signals in the content script. These concerns are orthogonal to the perception–action design and exist to make the prototype usable.

3.8 Security Concerns

The prototype in its current state has several vulnerabilities and must not be used in a public setting:

- The agentic system has no protection against prompt injections that can be present in page content. This can enable severely harmful behavior up to remote code execution, as the rule application sandbox can fetch data from any URL.
- The browser extension stores API keys in the browser's storage in plaintext.

Chapter 4

Methods

This section covers the data collection, processing pipelines, evaluation task synthesis, and the evaluation process itself.

4.1 DOM Compression Evaluation

To move from anecdotal DOM sizes to measurable compression, we assembled a corpus of real page snapshots and ran the perception pipeline on each one.

4.1.1 Source and filtering

We started from the top-domain list in the web-unpacked study [36], which annotates 116 high-traffic sites. We filtered out login- or payment-gated services and adult-industry domains, because manual evaluation might later involve inspecting page screenshots. We kept the top 25 remaining domains by rank. Commercial top-site lists can miss frequently visited pages and bias language-specific research [37], but the web-unpacked corpus offers richer metadata than rank-only lists for our sampling goals.

Several homepages needed manual URL adjustment before crawling: search engines and YouTube were pointed at result pages rather than landing pages, Wikipedia at the English homepage, and Pinterest at a browsable feed. We then crawled same-domain links from each seed URL using a headed browser, semi-automatically accepting cookie banners and manually passing captchas where required.

Further manual filtering removed dead or region-locked services, pages blocked by bot detection, legal pages (terms of service, privacy policy), regional duplicates, and sitemaps. From each surviving domain we sampled the homepage plus three randomly chosen internal pages.

4.1.2 Capture and quality control

Each selected URL was snapshotted with Playwright. For every page we stored:

1. `raw.html` — the full serialized DOM.
2. `visible.html` — the DOM after removing subtrees that are invisible by computed style (`display: none`, `visibility: hidden`, `opacity: 0`), matching the capture step used in the perception component.
3. A screenshot for optional human review.

Captchas and empty bot-rejection pages were handled in a semi-manual patch pass; snapshots that remained empty or unusable were dropped. The final corpus contains 73 pages across 25 domains.

4.1.3 Compression measurements

After capture, we ran the reproducible batch script in `experiments/dom-compression-analysis/` (see Section 8): for each snapshot it applies the production `get_map_of_dom` tool to `visible.html`, then counts characters and tokens (tiktoken `o200k_base`, matching TABLE 3.1). Aggregates and interpretation are reported in Section 5.1; those figures motivate the perception design in Section 3.4 and the context budget argument in Section 3.2.

4.2 Task Synthesis

Compression metrics alone do not define adaptation tasks. To evaluate the full agent, we synthesized user requests from the snapshot corpus using a Jobs-to-be-Done (JTBD) pipeline. This mirrors recent work on synthesizing NL edit datasets for web pages [38], but targets user-side adaptation prompts grounded in personas and jobs rather than developer-time HTML source edits.

We selected 10 snapshots from the corpus with a fixed random seed, excluding pages that failed quality checks. For each snapshot, a separate LLM session (Gemini 3.5 Flash) inspected the page DOM and produced, in three turns:

1. A list of user jobs on the page ("I want to…").
2. Three pairs of opposing user preferences — persistent traits rather than one-off edits.
3. Six concrete edit requests — one per preference side, labeled `1a` through `3b`.

Each request became a *seed sample*: a `task.json` describing the prompt and JTBD context, plus copies of the snapshot's `raw.html` and `visible.html`. This yields 60 seed tasks grounded in real pages and varied user intents. Downstream ablation

pipelines (Section 4.3) run different perception–action combinations on these seeds for controlled comparison.

4.3 Ablation Pipelines

To isolate the perception and action components, we run six agent configurations on the synthesized seed tasks:

TABLE 4.1
Agent ablation conditions — Perception and action combinations compared in the evaluation study.

Pipeline	Perception	Action
Baseline	full DOM	immediate patch
Engine only	full DOM	Rules Engine
Map only	DOM Compression	immediate patch
Full	DOM Compression	Rules Engine
Full (Sonnet)	DOM Compression	Rules Engine, Claude Sonnet 4.6

Baseline and map-only swap perception strategy while keeping a one-shot edit action. Engine-only and full swap persistence while keeping full-DOM or compact perception respectively. The full pipeline matches the production prototype; results are reported in the Results section.

4.3.1 Local inference hardware

Automated ablations were run on an NVIDIA H100 PCIe GPU (81,GiB VRAM), 16 server threads, and a 262,144-token context window with an 8,GiB prompt cache. We loaded Unsloth's `Qwen3.6-27B-UD-Q4_K_XL` GGUF weights.

We chose local inference for three practical reasons. First, the baseline condition reads the full visible DOM and routinely exceeds 100k tokens (TABLE 3.1); running every pipeline on the same long-context model avoids confounding API provider limits with architecture. Second, batch evaluation over dozens of samples is cheaper and more reproducible when inference stays on fixed hardware. Third, the production extension can call the same model family through a gateway, but the ablation study needed a stable backend for paired timing comparisons.

4.3.2 Human evaluation test stand

Subjective quality was collected with a small Next.js web application. The test stand is not part of the browser extension; it only replays archived evaluation artifacts.

For each trial the participant first sees the original-page screenshot and the adaptation prompt, then nine blinded pairwise screenshot comparisons per sample. Screenshots capture a fixed 1440×800 viewport and are not interactive — raters cannot scroll, reload, or verify that rules persist after navigation (Section 7.1). Pipelines are compared in a fixed set of pairs (original vs each treatment, baseline vs full, and the component ablations listed in Section 4.3); left and right positions are randomized per trial. We originally planned a five-point Likert scale, but piloting showed that raters rarely used the intermediate anchors. The deployed interface therefore asks for a simple preference: left better, right better, or similar (neutral).

4.3.3 Sample filtering and scope

The JTBD pipeline produced 60 seed tasks (Section 4.2). After automated runs and quality checks, 42 samples retained complete agent logs and screenshots for all ablation pipelines. Many candidates were dropped because archived HTML did not render faithfully in headless replay — broken asset URLs, bot walls, or empty post-capture pages — which made side-by-side screenshots unusable for human review.

Before the preference study we filtered again: only samples whose screenshots for every pipeline variant displayed the page content clearly entered the zip archive. That left 15 samples and 135 judgments (15 × 9 pairs). Statistical tests in Section 5.3 therefore have low power; we treat them as a preliminary human readout. Future work will improve snapshot renewal and rendering patches so more of the corpus becomes ratable.

4.4 AI Usage Disclosure

AI agents via Cursor Editor and Claude Code CLI were used in writing the prototype code. All generated code was fully reviewed and verified manually, and all architectural decisions were made explicitly by the authors. We used proprietary frontier models in the prototype evaluation; following [39], we treat this as justified because the contribution is the adaptation system rather than a model benchmark.

Chapter 5

Results

5.1 DOM compression

This section reports the DOM compression study in full. The corpus and capture protocol are defined in Section 4.1; Section 4.1.3 states only that we batch-ran the production compressor. The numbers below are what Section 3.2 and Section 3.4 cite when arguing that full-DOM reads are too large and that a compact map plus drill-down is required.

5.1.1 Corpus and procedure

We evaluated 73 page snapshots stored under `experiments/dom-compression-analysis/data/snapshots/` in the project repository (Section 8) — the same 25-domain sample as in Section 4.1.1 (homepage plus three internal URLs per domain, after manual quality filtering). For each snapshot id `NNN` we measured three HTML stages:

1. *Raw DOM* — serialized `raw.html` as captured by Playwright.

2. *Visible DOM* — `visible.html` after stripping subtrees hidden by computed style (`display: none`, `visibility: hidden`, `opacity: 0`), matching extension capture.
3. *Compressed DOM* — output of `get_map_of_dom` on that visible snapshot (all heuristics in Section 3.4.1).

Counts use tiktoken encoding `o200k_base` for tokens and UTF-8 byte length for characters. HTML comments introduced by the compressor (wrapper collapse, sibling truncation) are tracked separately as *comment overhead* inside the compressed file. The script writes per-page rows to `samples.csv` and corpus-level means/medians to `total.csv`.

5.1.2 Token counts

TABLE 5.1
Token counts after DOM compression — Token counts over 73 snapshots (`o200k_base`).

Stage	Mean	Median	Min	Max
Raw DOM	339,253	240,918	18,490	1,338,122
Visible DOM	162,806	107,851	7,188	1,028,594
Compressed DOM	11,135	6,613	522	107,851

Visible capture removes boilerplate that never renders: median size drops from 240,918 to 107,851 tokens (factor 2.2×). That step alone does not bring typical pages under a practical reasoning budget.

Compression removes non-structural markup, high-frequency classes, wrapper chains, and repeated siblings (Section 3.4.1). Median compressed DOM size is

6,613 tokens — **16.3×** smaller than the median visible DOM and 36.4× smaller than median raw. Mean compressed DOM size is 11,135 tokens versus 162,806 visible (14.6×). The distribution is skewed: the smallest compressed DOM is 522 tokens; the largest is 107,851 tokens, equal to the median *visible* page. On that outlier, heuristics barely shrink the tree, so the compressed DOM still occupies essentially a full visible DOM worth of context. Compression is therefore necessary for typical pages but not a universal fit guarantee.

TABLE 3.1 in Section 3.2 reproduces the raw and visible columns from this table; the compressed column appears only here.

5.1.3 Character counts and comment overhead

TABLE 5.2
Character counts after DOM compression — Character counts over 73 snapshots.

Stage	Mean	Median	Min	Max
Raw DOM	948,621	708,458	50,097	4,411,456
Visible DOM	421,952	295,185	21,212	2,353,194
Compressed DOM	35,730	21,424	1,854	332,518
Compressed DOM comments only	8,780	5,789	311	50,983

Character counts follow the same ordering as tokens. Median visible HTML is 295,185 characters; median compressed DOM is 21,424 (13.8× reduction). Annotation comments account for a median of 5,789 characters inside the compressed DOM — about 27% of median compressed file size — so a non-trivial share of the compact representation documents what was removed rather than live structure.

5.1.4 Implications for the prototype

Together, TABLE 5.1 and TABLE 5.2 show two separable wins: halving input by hiding invisible subtrees at capture time, then another order-of-magnitude reduction by structural compression before the first model call. That gap is what makes `get_map_of_dom` plus `show_in_dom` viable on median production pages while the read-all baseline in Section 3.2 still faces six-figure token medians. Selective drill-down remains necessary because the lossy map omits repeated list items and pared attributes; the agent recovers detail from the frozen snapshot only where needed (Section 3.4.2).

5.2 Automated ablation corpus

Across the 42 pipeline-complete samples:

- The baseline agent executed without context overflow on all 42 samples; visible DOM fit a 202,144-token budget on 33/42 (78.6%).
- Every baseline run produced an edited `index.html`; the full pipeline (`5-full`) also completed on 42/42 samples.

Processing time is summed `elapsed_s` per sample from `agent.log` (model inference only). Friedman test across all five timed pipelines: $n = 42$, $\chi^2 = 116.3$, $p = 3.3 \times 10^{-24}$.

TABLE 5.3
Median inference time by ablation pipeline — Median API inference time per sample over 42 tasks (seconds).

Pipeline	Median	Mean	Min	Max
Baseline	162.8 s	202.5 s	14.2 s	513.6 s
Engine only	63.4 s	74.4 s	6.7 s	155.1 s

Pipeline	Median	Mean	Min	Max
Map only	82.3 s	116.8 s	13.9 s	361.2 s
Full	34.4 s	37.6 s	9.4 s	119.8 s
Full (Sonnet)	28.2 s	33.1 s	14.1 s	78.5 s

Paired Wilcoxon tests (same 42 samples): baseline vs full — median 162.8 s vs 34.4 s, speedup ×4.74, $p = 4.5 \times 10^{-13}$, full faster on 42/42; baseline vs engine-only — ×2.57, $p = 1.4 \times 10^{-12}$; baseline vs map-only — ×1.98, $p = 6.0 \times 10^{-6}$; map-only vs full — ×2.40, $p = 6.4 \times 10^{-12}$; engine-only vs full — ×1.85, $p = 1.1 \times 10^{-8}$; baseline vs full-sonnet — ×5.77, $p = 4.5 \times 10^{-12}$; full vs full-sonnet — ×1.22, $p = 0.080$ (not significant at $\alpha = 0.05$).

5.3 Pairwise preference evaluation

Human judgments cover 15 samples (135 comparisons) — a small set with limited statistical power (Section 7.1). Task completion vs original counts only decisive outcomes (excluding ties) from static screenshots; it does not verify reload persistence or interactive behavior:

TABLE 5.4
Screenshot task completion vs. original page — Decisive win / loss / tie counts ($n =$ 15 samples per row).

Treatment	Wins	Losses	Ties	Decisive win rate
Baseline	12	0	3	100% (12/12)
Full	11	0	4	100% (11/11)
Full (Sonnet)	11	0	4	100% (11/11)

Exact binomial sign tests for baseline and full vs original are significant at $\alpha = 0.05$ ($p < 0.001$ for both). McNemar tests on discordant baseline-vs-full outcomes when both are compared to original report zero discordant pairs.

Head-to-head quality (full vs baseline on the same samples): 4 wins, 6 losses, 5 ties; decisive win rate for full = 40% (4/10); exact binomial $p = 0.754$; Wilcoxon on signed preference scores $p = 0.625$. Quality parity between full and baseline is therefore inconclusive — the speed gains in Section 5.2 are not matched by a significant human preference for either pipeline.

Component ablation (decisive win rate for the named pipeline):

TABLE 5.5
Human preference scores for component ablation — Decisive preferences on 15 rated samples.

Comparison	Wins	Losses	Ties
Engine only vs baseline	2	7	6
Map only vs baseline	5	1	9
Full vs engine only	4	1	10
Full vs map only	2	5	8
Full vs baseline	4	6	5

None of the component sign tests reach $p < 0.05$ at $n = 15$.

5.4 Extension cases outside the corpus

The evaluation protocol is necessary for comparison, but it does not show what using Internet Shaper on a live tab feels like. We therefore include three informal

examples from real browser-extension sessions on production sites — the same tool a user would run day to day.

Each figure pairs the page before the user submits a prompt with the state after stored rules re-apply. They are illustrative, not part of the 15-sample preference study in Section 5.3.

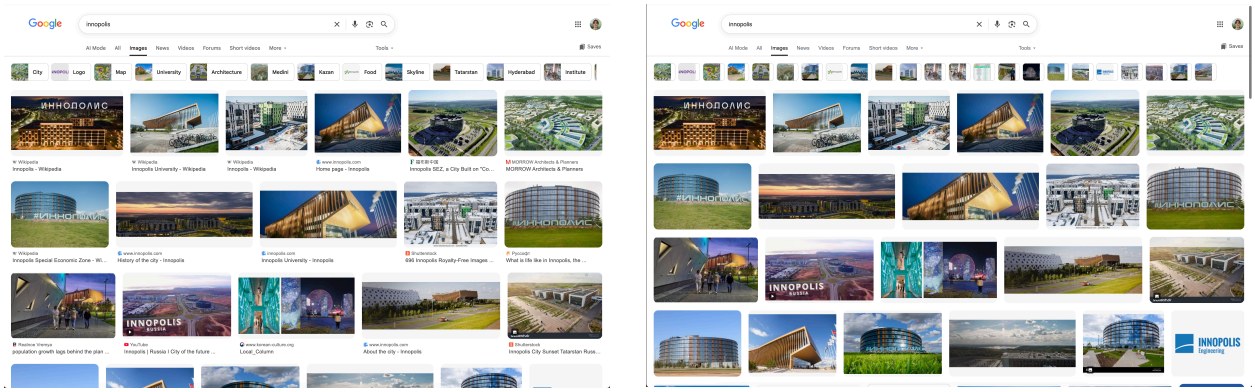

Fig. 5.1. System in action, example 1. Google Images — Prompt: «remove the text under images».

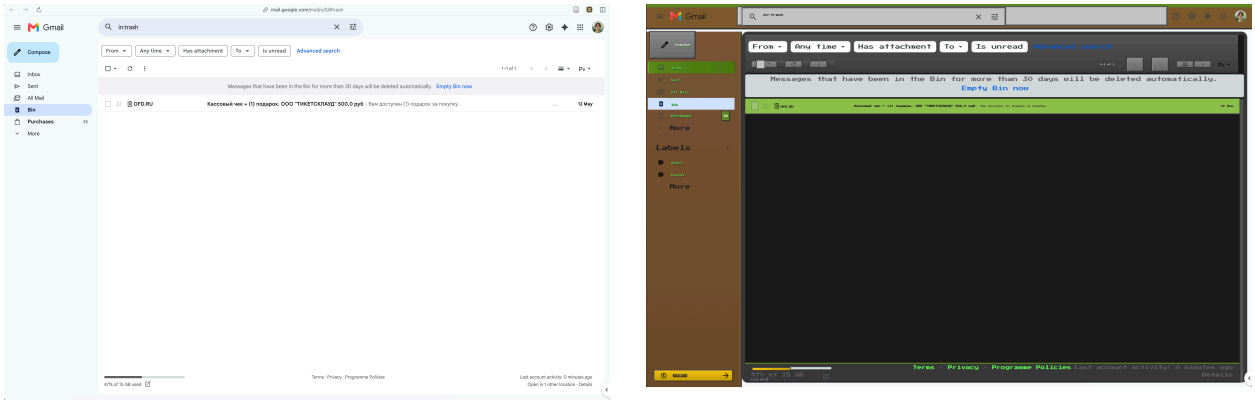

Fig. 5.2. System in action, example 2. Gmail — Prompt: «style the page in Minecraft theme».

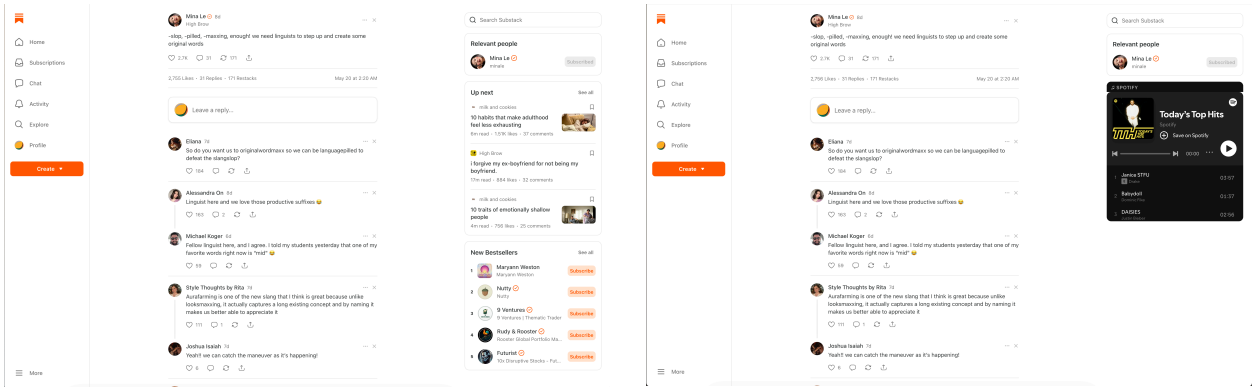

Fig. 5.3. System in action, example 3. Substack — Prompt: «replace the Up next block with an audio player from Spotify».

Chapter 6

Discussion

The evaluation corpus ended up much smaller than we planned: 60 synthesized seeds, 42 machine-runnable samples, and only 15 that survived screenshot quality filtering for human review. Power for pairwise tests is therefore limited, and non-significant component contrasts should be read as inconclusive rather than as evidence of parity.

Within that constraint, both baseline and full beat the original on decisive screenshot judgments (TABLE 5.4), but many comparisons were ties and the raters could not interact with the pages — so task-completion claims should be read as weak evidence of visual improvement, not verified functional success. When baseline and full are compared directly, full wins 40% of decisive pairs and loses 60% — quality parity remains inconclusive ($p = 0.754$). The compact perception plus rules engine therefore delivers large inference-time savings (TABLE 5.3) without a demonstrated human quality advantage over the read-all baseline.

Processing times show a clearer pattern (TABLE 5.3). The full pipeline mediates 4.7× faster inference than baseline on the same 42 tasks, with map-only and

engine-only ablations in between. Because rules persist in extension storage, those minutes of model time are spent once per adaptation goal rather than on every revisit.

The live extension cases (Section 5.4) illustrate behaviors we did not encode explicitly. In informal trials the agent has called third-party HTTP APIs when asked to add decorative content, composed multi-rule layout refactors (for example a Substack article grid with roughly seven coordinated rules), and collapsed large restructuring tasks into a single `main`-scoped rule with internal control flow. We did not run the same prompts through the baseline pipeline or score how often comparable patterns appear without DOM compression and persistent rules; attributing them uniquely to Internet Shaper would require a follow-up controlled study.

6.1 Emergent behaviors

Reported informally during extension use (not part of the 15-sample preference study):

- When asked to «add a cat to the page», the agent fetched a random image from cataas.com and injected it into the DOM.
- Beyond trivial hide/show requests, generated rule logic often uses variables, conditionals, and child-node properties.
- Grid-style layout requests (Substack article list) triggered coordinated changes to cards, thumbnails, and menus, not a single CSS tweak.
- Large structural moves sometimes become one rule on `main` rather than many element-specific rules.

Chapter 7

Conclusion

This thesis presented Internet Shaper, a browser-extension-hosted agent that adapts third-party web pages from natural language without site cooperation. The technical contributions are (1) DOM compression with selective drill-down for scalable perception, (2) a persistent rules engine for expressive, reload-safe action, and (3) a JTBD-based pipeline for synthesizing grounded adaptation tasks on real page snapshots.

Empirically, compression reduces median visible DOM size by roughly 16× on a 73-page corpus. On 42 automated samples the full pipeline runs about five times faster than a full-DOM baseline. In a preliminary human study ($n = 15$) both pipelines visually outperformed the original on decisive screenshot judgments, but quality parity between baseline and full was inconclusive and many comparisons were ties. Three live-site case screenshots demonstrate non-trivial adaptations outside the formal dataset.

The human evaluation is preliminary because rendering failures forced heavy sample filtering. Future work will harden snapshot replay, expand the ratable set

toward statistically powered inference, and measure whether emergent behaviors such as API calls and multi-rule layout refactors appear in baseline conditions as well.

7.1 Limitations

1. Internet Shaper operates on a single screen at a time: the agent has no multi-screen context and cannot construct multi-step or app-wide flows.
2. Rules are scoped to the hostname with no URL path matching, so adaptations intended for one page may apply across the whole domain (Section 3.5.1).
3. The human evaluation is preliminary: only 15 samples survived screenshot filtering, judgments are static viewport PNGs with no reload or interaction, many comparisons were ties, and baseline-vs-full quality parity is inconclusive ($p = 0.754$; Section 5.3).
4. Adaptation prompts were LLM-synthesized rather than collected from real users, and automated ablations replay archived HTML in batch scripts — approximating but not identical to live extension use (Section 4.2, Section 4.3).
5. The prototype is not production-ready: it is vulnerable to prompt injection via page content and stores API keys in plaintext (Section 3.8).

Chapter 8

Data and Code Availability

All source code, experiment pipelines, evaluation datasets, agent logs, analysis scripts, and supplementary materials for this thesis are available in the public repository github.com/andrewlevada/internet-shaper-thesis-2026.

Bibliography cited

[1] P. A. Hancock, A. A. Pepe, and L. L. Murphy, “Hedonomics: The Power of Positive and Pleasurable Ergonomics,” *Ergonomics in Design*, vol. 13, no. 1, pp. 8–14, Jan. 2005, doi: 10.1177/106480460501300104.

[2] N. S. M. Yusop, J. Grundy, J.-G. Schneider, and R. Vasa, “A revised open source usability defect classification taxonomy,” *Information and Software Technology*, vol. 128, p. 106396, Dec. 2020, doi: 10.1016/j.infsof.2020.106396.

[3] R. G. Timms, “All ‘Dark patterns’ Are ‘Hostile patterns’: A Hostility Framework for Understanding Problematic Digital Interfaces,” *Ethics and Information Technology*, vol. 27, no. 4, p. 57, Oct. 2025, doi: 10.1007/s10676-025-09856-z.

[4] L. A. Baroni and R. Pereira, “Deceptive Patterns under a Sociotechnical View,” in *Proceedings of the XXIII Brazilian Symposium on Human Factors in Computing Systems*, in IHC '24. New York, NY, USA: Association for Computing Machinery, Dec. 2024, pp. 1–13. doi: 10.1145/3702038.3702081.

[5] M. Potel-Saville and M. Francois, *From Dark Patterns to Fair Patterns? Usable Taxonomy to Contribute Solving the Issue with Countermeasures*. 2023.

[6] G. Litt, D. Jackson, T. Millis, and J. Quaye, “End-user software customization by direct manipulation of tabular data,” in *Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, in Onward! 2020. New York, NY, USA: Association for Computing Machinery, Nov. 2020, pp. 18–33. doi: 10.1145/3426428.3426914.

[7] K. Katongo, G. Litt, and D. Jackson, “Towards End-User Web Scraping for Customization,” in *Companion Proceedings of the 5th International Conference on the Art, Science, and Engineering of Programming*, in Programming '21. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 49–59. doi: 10.1145/3464432.3464437.

[8] NNg, “Generative UI and Outcome-Oriented Design.” [Online]. Available: https://www.nngroup.com/articles/generative-ui/

[9] “Introducing A2UI: An open project for agent-driven interfaces- Google Developers Blog.” [Online]. Available: https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces/

[10] K. Lee, “Towards a Working Definition of Designing Generative User Interfaces | Companion Publication of the 2025 ACM Designing Interactive Systems Conference.” [Online]. Available: https://dl.acm.org/doi/10.1145/3715668.3736365

[11] E. Kim, S. S. Chowdhury, H. Song, and B. Suh, “In-Situ Adaptive Interfaces for Online Browsing: Design Dimensions for Intent-Responsive Automation and

User Control," in *Proceedings of the 31st International Conference on Intelligent User Interfaces*, in IUI '26. New York, NY, USA: Association for Computing Machinery, Mar. 2026, pp. 174–196. doi: 10.1145/3742413.3789092.

[12] Y. Lu, Y. Yang, Q. Zhao, C. Zhang, and T. J.-J. Li, "AI Assistance for UX: A Literature Review Through Human-Centered AI." [Online]. Available: http://arxiv.org/abs/2402.06089

[13] B. Min, A. Chen, Y. Cao, and H. Xia, "Malleable Overview-Detail Interfaces," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, in CHI '25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 1–25. doi: 10.1145/3706598.3714164.

[14] Y. Cao, P. Jiang, and H. Xia, "Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, in CHI '25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 1–20. doi: 10.1145/3706598.3713285.

[15] B. Wang, G. Li, and Y. Li, "Enabling Conversational Interaction with Mobile UI using Large Language Models," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, in CHI '23. New York, NY, USA: Association for Computing Machinery, Apr. 2023, pp. 1–17. doi: 10.1145/3544548.3580895.

[16] K. Tanner, N. Johnson, and J. A. Landay, "Poirot: A Web Inspector for Designers," in *Proceedings of the 2019 CHI Conference on Human Factors*

in Computing Systems, in CHI '19. New York, NY, USA: Association for Computing Machinery, May 2019, pp. 1–12. doi: 10.1145/3290605.3300758.

[17] T. Long and W. Peng, "PortfolioMentor: Multimodal Generative AI Companion for Learning and Crafting Interactive Digital Art Portfolios." [Online]. Available: http://arxiv.org/abs/2311.14091

[18] D. Jeong, S. Byun, K. Son, D. H. Kim, and J. Kim, "CANVAS: A Benchmark for Vision-Language Models on Tool-Based User Interface Design." [Online]. Available: http://arxiv.org/abs/2511.20737

[19] K. Katongo, G. Litt, K. Jin, and D. Jackson, "Joker: A Unified Interaction Model for Web Customization," in *Proceedings of the 7th Workshop on Live Programming (LIVE)*, 2021.

[20] J. Lin, J. Wong, J. Nichols, A. Cypher, and T. A. Lau, "End-User Programming of Mashups with Vegemite," in *Proceedings of the 2009 International Conference on Intelligent User Interfaces*, ACM, 2009, pp. 97–106.

[21] D. F. Huynh, R. C. Miller, and D. R. Karger, "Enabling Web Browsers to Augment Web Sites' Filtering and Sorting Functionalities," in *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2006. doi: 10.1145/1166253.1166274.

[22] O. Díaz, C. Arellano, and M. Azanza, "A Language for End-User Web Augmentation: Caring for Producers and Consumers Alike," *ACM Transactions on the Web*, vol. 7, no. 2, pp. 8:1–8:40, 2013, doi: 10.1145/2460383.2460388.

[23] O. Díaz, I. Aldalur, C. Arellano, H. Medina, and S. Firmenich, “Web Mashups with WebMakeup,” *Communications in Computer and Information Science*, vol. 591. Springer, pp. 82–97, 2016. doi: 10.1007/978-3-319-28727-0_6.

[24] I. Aldalur, A. Perez, and F. Larrinaga, “MAWA: A Browser Extension for Mobile Web Augmentation,” in *Human-Computer Interaction – INTERACT 2021*, in Lecture Notes in Computer Science, vol. 12935. Springer, 2021, pp. 221–242. doi: 10.1007/978-3-030-85610-6_14.

[25] M. Nebeling, M. Speicher, and M. Norrie, “CrowdAdapt: Enabling Crowdsourced Web Page Adaptation for Individual Viewing Conditions and Preferences,” June 2013, p. 23. doi: 10.1145/2494603.2480304.

[26] V. F. d. Santana and M. C. C. Baranauskas, “Continuous Web Personalization Using Selector-Template Pairs,” in *Proceedings of the 16th Web for All Conference (W4A 2019)*, ACM, 2019.

[27] T. S. Kim, D. Choi, Y. Choi, and J. Kim, “Stylette: Styling the Web with Natural Language,” in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, in CHI '22. New York, NY, USA: Association for Computing Machinery, Apr. 2022, pp. 1–17. doi: 10.1145/3491102.3501931.

[28] A. Sridhar, R. Lo, F. Xu, H. Zhu, and S. Zhou, “Hierarchical Prompting Assists Large Language Model on Web Navigation,” July 2023. doi: 10.18653/v1/2023.findings-emnlp.685.

[29] L. Zheng, R. Wang, X. Wang, and B. An, “Synapse: Trajectory-as-Exemplar Prompting with Memory for Computer Control.” [Online]. Available: http://arxiv.org/abs/2306.07863

[30] M. Enomoto, R. Obara, H. Zhang, and M. Oyamada, “Read More, Think More: Revisiting Observation Reduction for Web Agents.” [Online]. Available: https://arxiv.org/abs/2604.01535v1

[31] L. Ning *et al.*, “A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models.” [Online]. Available: http://arxiv.org/abs/2503.23350

[32] J. Zhang, K. Chen, Z. Lu, E. Zhou, Q. Yu, and J. Zhang, “Prune4Web: DOM Tree Pruning Programming for Web Agent,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 41, pp. 34710–34718, Mar. 2026, doi: 10.1609/aaai.v40i41.40772.

[33] J. Huang, “A Lightweight DOM-Aware Summarization Method for Low-Cost LLM-Based Web Page Understanding,” in *2025 7th International Academic Exchange Conference on Science and Technology Innovation (IAECST)*, Dec. 2025, pp. 1–5. doi: 10.1109/IAECST68792.2025.11414913.

[34] D. Zan *et al.*, “Large Language Models Meet NL2Code: A Survey,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds., Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 7443–7464. doi: 10.18653/v1/2023.acl-long.411.

[35] "Tool use with Claude." [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

[36] H. S. Xavier, "The Web unpacked: a quantitative analysis of global Web usage," in *Proceedings of the 20th International Conference on Web Information Systems and Technologies*, 2024, pp. 183–190. doi: 10.5220/0012905900003825.

[37] T. Alby and R. Jäschke, "Analyzing the Web: Are Top Websites Lists a Good Choice for Research?," in *Linking Theory and Practice of Digital Libraries*, G. Silvello, O. Corcho, P. Manghi, G. M. Di Nunzio, K. Golub, N. Ferro, and A. Poggi, Eds., Cham: Springer International Publishing, 2022, pp. 11–25. doi: 10.1007/978-3-031-16802-4_2.

[38] T. H. Dang, J. Xiao, and Y. Huo, "Envisioning Future Interactive Web Development: Editing Webpage with Natural Language." [Online]. Available: http://arxiv.org/abs/2510.26516

[39] A. Palmer, N. A. Smith, and A. Spirling, "Using proprietary language models in academic research requires explicit justification," *Nature Computational Science*, vol. 4, no. 1, pp. 2–3, Jan. 2024, doi: 10.1038/s43588-023-00585-1.